

Задание №1. Разработка и анализ прототипа backend-сервиса MediConnect с помощью вайб-кодинга

Студент: Турищев Артём Игоревич

Группа: K3340

Репозиторий: [art020205/pris_2_1](https://github.com/art020205/pris_2_1)

T3 / промпт: файл [task.md](#) в репозитории

Чат с БЯМ: использовался агентный чат Codex. Публичной ссылкой на чат поделиться нельзя, так как данный агентный чат не поддерживает возможность шаринга. Использованный промпт, то есть техническое задание, сохранён в репозитории в файле [task.md](#).

Цель работы

Целью работы является разработка и анализ прототипа backend-сервиса MediConnect с помощью средств вайб-кодинга на базе большой языковой модели. Сервис предназначен для безопасного обмена медицинскими документами между пациентами, врачами и администратором.

Задачи работы

1. Проработать структуру backend-сервиса, основные сущности данных и сценарии взаимодействия пользователей.
2. Реализовать MVP приложения с использованием FastAPI, MongoDB, MinIO/S3-compatible storage, JWT-аутентификации и Docker Compose.
3. Обеспечить разграничение прав доступа для ролей patient, doctor и admin.
4. Реализовать загрузку, хранение и скачивание медицинских документов только через backend без выдачи публичных ссылок на S3.
5. Добавить аудит ключевых действий пользователей.
6. Провести базовое тестирование и проанализировать качество полученного результата.

Краткое описание реализованного сервиса

В рамках работы был реализован MVP backend-платформы MediConnect. Приложение предоставляет REST API для регистрации и авторизации пользователей, управления профилем, загрузки медицинских документов, выдачи и отзыва доступа врача к документам пациента, скачивания файлов через backend, просмотра организаций и административного аудита.

Основной стек проекта:

- Python 3.11;
- FastAPI;
- MongoDB;
- Motor;
- MinIO как S3-compatible storage;
- JWT Bearer auth;
- Pydantic v2;

- Docker Compose;
- pytest.

Проект разворачивается одной командой:

```
docker compose up --build
```

После запуска API доступен по адресу <http://localhost:8000>, Swagger UI доступен по адресу <http://localhost:8000/docs>.

Основные сценарии работы

В сервисе реализованы следующие пользовательские сценарии:

1. Регистрация пациента и врача.
2. Автоматическое создание администратора при старте приложения.
3. Авторизация пользователя и получение JWT access token.
4. Загрузка пациентом собственного медицинского документа.
5. Проверка, что врач не видит документы без выданного доступа.
6. Выдача пациентом доступа врачу к документу.
7. Просмотр и скачивание врачом доступного документа через backend.
8. Отзыв пациентом доступа.
9. Проверка, что после отзыва врач снова не видит документ.
10. Просмотр администратором пользователей, документов и audit logs.

Файлы не конвертируются к единому формату и сохраняются в MinIO как есть. В MongoDB сохраняются только метаданные документа, информация о файле, правах доступа и аудит.

Тестирование

Для проверки результата были добавлены базовые pytest-тесты:

- тест регистрации, входа и получения текущего пользователя;
- тест регистрации пациента с датой рождения;
- тест загрузки и скачивания документа пациентом;
- тест проверки доступа врача к документу через active grant.

Во время тестирования был обнаружен баг при попытке регистрации нового пользователя с датой рождения. Ошибка возникала из-за того, что MongoDB/BSON не поддерживает прямое сохранение объекта `datetime.date`. Агент сразу исправил проблему: дата рождения теперь преобразуется в BSON-совместимый `datetime` при сохранении, а в ответах API снова сериализуется в формат `YYYY-MM-DD`. После исправления был добавлен регрессионный тест, подтверждающий корректную регистрацию пользователя с `birth_date`.

Итоговый результат локального тестирования:

4 passed

Анализ результата

Полученный с помощью вайб-кодинга backend-прототип можно считать успешным для уровня MVP. Сервис покрывает основные требования технического задания: реализована ролевая модель, JWT-аутентификация, хранение файлов отдельно от базы данных, приватный MinIO bucket, доступ к скачиванию только через backend, soft delete документов и audit logs для значимых действий. Структура проекта разделена по слоям: конфигурация, база данных, схемы, сервисы и API-маршруты. Это делает код достаточно читаемым и расширяемым.

Качество кода в целом соответствует прототипу: логика доступа вынесена в отдельный сервис, конфигурация задаётся через переменные окружения, запуск приложения описан через Docker Compose. При этом есть направления для дальнейшего улучшения. Например, часть операций с S3 выполняется синхронно через boto3 внутри аsync-приложения; для MVP это допустимо, но при росте нагрузки лучше вынести такие операции в threadpool или использовать аsync-compatible клиент. Также можно усилить валидацию входных данных, добавить пагинацию списков, более подробные интеграционные тесты с реальными MongoDB и MinIO, а также структурированное логирование.

С точки зрения производительности сервис должен быть достаточен для демонстрационного сценария и малой нагрузки. FastAPI и Motor позволяют эффективно обрабатывать асинхронные запросы к MongoDB, а MinIO подходит для хранения больших медицинских файлов вне базы данных. Потенциальные узкие места связаны с загрузкой файла целиком в память при upload и синхронным S3-клиентом. Для промышленной версии стоило бы реализовать потоковую загрузку, ограничение типов и размеров файлов на уровне middleware, мониторинг и более строгую политику безопасности.

Развёртывание сервиса получилось удобным: весь MVP запускается одной командой через Docker Compose, включая API, MongoDB, MinIO и инициализацию private bucket. Это позволяет быстро проверить основные пайплайны на локальной машине или на сервере с Docker. Для production-развёртывания потребуются отдельные настройки секретов, TLS, резервное копирование MongoDB и MinIO, а также более строгая настройка CORS и сетевого доступа.

Рефлексивное заключение

В результате работы удалось получить рабочий backend-прототип сервиса для обмена медицинскими документами. Вайб-кодинг оказался полезен для быстрой сборки структуры проекта, реализации типовых API-сценариев и написания базовых тестов. Особенно полезным было то, что агент мог не только генерировать код, но и сразу запускать проверки, находить ошибки и исправлять их.

Основная проблема в ходе выполнения была связана с несовместимостью типа `datetime.date` и BSON при регистрации пользователя. Этот баг проявился уже после первичной реализации, при реальном тестировании сценария регистрации. Ошибка была оперативно исправлена, а тестовая база дополнена отдельным регрессионным тестом. Также в процессе работы проявилась важность фиксации совместимых версий зависимостей, например для `passlib` и `bcrypt`.

Из данной работы удалось лучше понять, что вайб-кодинг хорошо ускоряет создание MVP, но не отменяет необходимости тестирования, анализа архитектурных решений и ручной проверки edge cases.

Большая языковая модель может быстро построить основу сервиса, однако итоговое качество зависит от корректности технического задания, полноты тестовых сценариев и последующей инженерной доработки.